Report for Learing Agent Submission

Nuo Chen (A0314255L)
Chang Xiaoyin (A0318376W)
Chen Zihan (A0240951J)

November 7, 2025

1 Code Organization

`/learning_agent`

- `train_ppo/train_sac.py` – Trains the agent using PPO or SAC.
- `ppo_agent/sac_agent.py` – Implements PPO or SAC update logic.
- `replay_buffer.py` – Stores and samples transitions.

2 Code logic

2.1 ReplayBuffer

store()- records each transition $(s, a, r, s', \texttt{done})$ —generated by the environment at every time step—into a cyclic first-in-first-out (FIFO) replay buffer.

sample()– Constructs a batch of valid, temporally continuous, and non-overlapping sequences by selecting indices `seq_indices` = `np.arange`$(i - L + 1, i + 1)$, where i is the terminal index of a trajectory segment and L is the sequence length. These sequences are then converted into tensors for training the LSTM-based SAC networks.

2.2 Network Update Based On Target Function

- PPO-clip

  Actor and Critic Loss Function introduction (why designed this way), components (e.g. target Q in critic), and short introduction about tool functions used to get the components

$$\arg\max_{\theta} E_{s\sim\nu^{\pi_{\theta_k}},\, a\sim\pi_{\theta_k}(\cdot|s)}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\pi_{\theta_k}}(s,a), \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right)A^{\pi_{\theta_k}}(s,a)\right)\right] \tag{1}$$

  Tool function: The function $\text{clip}(r_t, 1-\epsilon, 1+\epsilon)$ constrains the probability ratio within a trust region, preventing occasional high-advantage samples from pushing the policy too far. Gradients outside this range are truncated to zero, leading to more stable training and indirectly reducing value-estimation noise.

  The function `generalized_advantage_estimation()` computes a discounted and smoothed sum of multi-step temporal-difference errors to balance bias and variance, thereby producing stable advantage estimates for PPO policy updates.

  The function `PPOAgent.update()` computes the objective function of PPO. It first calculates the *policy loss* , then optionally clips the value function according to the configuration and obtains the *value loss* . Finally, it combines all parts into the total PPO loss, followed by backpropagation to update the network parameters.

- **SAC**

  Code Foundation: The Soft Actor-Critic (SAC) algorithm optimizes both the policy (actor) and value function (critics) while maximizing entropy to encourage exploration.

$$E_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_{\theta_i}(s,a)-\left(r+\gamma\, E_{a'\sim\pi_\phi(\cdot|s')}\left[\min_{j=1,2} Q_{\bar{\theta}_j}(s',a')-\alpha\log\pi_\phi(a'|s')\right]\right)\right)^2\right] \tag{2}$$

$$E_{s\sim\mathcal{D}}\left[E_{a\sim\pi_\phi(\cdot|s)}\left[\alpha\log\pi_\phi(a|s)-Q_{\theta_1}(s,a)\right]\right] \tag{3}$$

  Tool Functions:

  - `update()`: performs one gradient update step for the actor, critics, and the temperature parameter α, forming the core training loop of the SAC agent. It samples a batch from the replay buffer, computes target Q-values, and updates the networks. When automatic entropy tuning is enabled, α is updated by minimizing

$$\mathcal{L}_\alpha = -\log\alpha\cdot(\log\pi(a|s)+\mathcal{H}_{target}),$$

    which adjusts the policy entropy toward the target value. The function returns the losses for logging and analysis.
  - `compute_grad_norm()`: A diagnostic utility to monitor training stability and detect issues like gradient explosion.

2.3 Train Agent

The training script first fills the buffer with random experiences until reaching the **minimum size of 10,000** samples. Then, at each step, a sequence of length *seq_len* is formed and passed to the network, where the agent selects an action via *select_action()*. The environment executes the action, and the resulting transition is stored in the buffer. The networks are updated every *update_frequency* steps, while losses are logged and the agent is periodically evaluated using **evaluate_agent()** function.

3 Libraries used

PyTorch was used to implement and train the neural networks, providing a flexible framework with dynamic computation graphs and seamless GPU acceleration.

NumPy was employed for efficient numerical computations, particularly for array operations and matrix manipulations that support data preprocessing and environment interactions.

4 Reflections/Lessons Learned

- Handling sequential data requires careful design: backward look-up must be implemented, action-state alignment ensured, and a `state_seq_array` maintained during rollouts to enable the agent to generate actions correctly.
- Building the agent clarified the tight coupling between actor and critic—they mutually depend on each other's outputs and co-evolve through training. Furthermore, integrating the SAC agent with the Gym environment and neural networks into a unified training loop provided a clear, end-to-end understanding of the reinforcement learning pipeline.
- We gained a clear understanding of proper evaluation protocols in reinforcement learning, including deterministic action selection and consistent environment seeding for fair performance assessment.

5 AI Usage

ChatGPT was used to help with code and report polishing.